Set of **20 Python string assignment questions**—first **10 beginner**, then **10 intermediate**—each with a **concise hint** to guide learners without giving away full solutions. I've kept them practical and classroom-ready.

Beginner Level (Strings Fundamentals)

1. **Length of a String**
   Write a program that reads a string and prints its length.
   - Input: "hello world" → Output: 11
     Hint: Use len(s).
2. **Uppercase & Lowercase**
   Convert the input string to uppercase and lowercase.
   - Input: "Python3" → Output: "PYTHON3", "python3"
     Hint: Methods: s.upper(), s.lower().
3. **Count a Character**
   Count how many times a given character appears in a string (case-sensitive).
   - Input: "banana", "a" → Output: 3
     Hint: Use s.count(ch).
4. **First & Last Character**
   Print the first and last character of a string; handle empty input.
   - Input: "drawer" → Output: First: d, Last: r
     Hint: Check empty with if not s; index via s[0] and s[-1].
5. **Check Substring Presence**
   Check if a substring exists in a string.
   - Input: "data science", "science" → Output: True
     Hint: Use the in operator: sub in s.
6. **Slice a String**
   Print a substring from index start to end (exclusive).
   - Input: "programming", 3, 8 → Output: "gramm"
     Hint: Use slicing: s[start:end].
7. **Reverse a String**
   Reverse the string.
   - Input: "Python" → Output: "nohtyP"
     Hint: Slicing trick: s[::-1].
8. **Replace Substring**
   Replace all occurrences of a word with another (case-sensitive).
   - Input: "I love apples. Apples are great!", "apples", "oranges"
   - Output: "I love oranges. Apples are great!"
     Hint: s.replace(old, new) replaces exactly matching cases.
9. **Split and Join**
   Split a sentence on spaces and join with -.

- Input: "split this sentence" → Output: "split-this-sentence"
  Hint: s.split() then "-".join(words).

10. **Strip Whitespace**
    Remove leading and trailing spaces.
    - Input: " padded text " → Output: "padded text"
      Hint: Use s.strip().

Intermediate Level (More Involved String Tasks)

1. **Count Vowels & Consonants**
   Count vowels and consonants (letters only; ignore digits/punctuation).
   - Input: "Hello, World! 123" → Output: Vowels: 3, Consonants: 7
     Hint: Iterate characters; check ch.isalpha(); membership test like ch.lower() in "aeiou".
2. **Palindrome Check (Ignore Case & Non-alphanumerics)**
   Determine if a string is a palindrome ignoring case and non-alphanumeric characters.
   - Input: "A man, a plan, a canal: Panama!" → Output: True
     Hint: Normalize with ''.join(ch.lower() for ch in s if ch.isalnum()); compare to its reverse.
3. **Title Case (Manual)**
   Convert a sentence to title case **without** using .title().
   - Input: "hELLO wORLD from PYTHON" → Output: "Hello World From Python"
     Hint: Split into words; for each word: word[0].upper() + word[1:].lower() (guard empty words).
4. **Find All Indices of a Substring (Allow Overlaps)**
   Return a list of starting indices where a substring occurs.
   - Input: s="aaaa", sub="aa" → Output: [0, 1, 2]
     Hint: Loop i from 0 to len(s) - len(sub); compare slices s[i:i+len(sub)].
5. **Character Frequency Dictionary**
   Build a frequency dictionary for characters (case-insensitive, skip spaces).
   - Input: "Baa Baa Black Sheep"
   - Output (order may vary): {'b':3,'a':5,'l':1,'c':1,'k':1,'s':1,'h':1,'e':3,'p':1}
     Hint: Iterate for ch in s.lower(): and if ch != ' ': then count with a dict; dict.get(ch, 0)+=1.
6. **Anagram Checker**
   Check if two strings are anagrams (ignore spaces, punctuation, and case).
   - Input: "Listen", "Silent" → Output: True
     Hint: Normalize to letters with ch.isalpha() and lower(), then compare sorted(s1) vs sorted(s2) or frequency dicts.

7. **Compress Repeated Characters (RLE-lite)**
   Compress runs of the same character as <char><count>.
   - Input: "aaabbcaaaa" → Output: "a3b2c1a4"
     Hint: Track current char and run length; flush when char changes or at the end.
8. **Longest Word in a Sentence**
   Find the longest word; if multiple, return the first. Consider words as alphabetic sequences.
   - Input: "Find the longest_word here!" → Output: "longest"
     Hint: Filter to letters using ''.join(ch for ch in token if ch.isalpha()); track max by length.
9. **Remove Duplicate Characters but Keep Order**
   Remove duplicates while preserving the first occurrence order.
   - Input: "banana" → Output: "ban"
     Hint: Maintain a seen set; build result by adding chars not in seen.
10. **Mask Email Username**
    Mask all but the first and last character of the username with *; keep domain intact.
    - Input: "john.doe@example.com" → Output: "j******e@example.com"
      Hint: Split on '@'; for the left part, if length ≥ 2, keep first and last and replace middle with '*' * (len-2); if shorter, handle edge cases.